RIDHIMA WORLD SCHOOL

INFORMATICS PRACTICES

(Code 065)

IP

PRACTICAL FILE

SESSION 2026 - 2027

17 PYTHON PROGRAMS | 6 SQL COMMANDS

NAME	**Aditya Shakya**
CLASS	**XI - A**
SECTION	**A**
ROLL NUMBER	**2**

SUBMITTED TO

Mr. Arya Sir
Department of Computer Science
Ridhima World School

SUBMITTED BY

Aditya Shakya
Class XI - A, Section A
Roll No. 2

`github.com/zionixxdev/xi-ip-practicals`
GitHub Repository for Practicals


```
>>> print('Informatics Practices - Practical File')
```

CERTIFICATE

This is to certify that **Aditya Shakya**, a student of **Class XI - A**, **Section A**, **Roll No. 2** of **Ridhima World School**, has satisfactorily completed the practical work in **Informatics Practices (Code 065)** prescribed by the Central Board of Secondary Education for the academic session **2026 - 2027**, under my supervision and guidance.

The file contains seventeen programs of Python programming and six commands of SQL, along with their outputs and conclusions. The work has been carried out by the student himself and has been checked and found to be correct.

Date : _____

Place : _____

_____
Mr. Arya Sir
(Internal Examiner)

ACKNOWLEDGEMENT

a few words of thanks

I express my sincere gratitude to my teacher, **Mr. Arya Sir**, for the valuable guidance and constant support that made this practical work possible, and to the Principal of **Ridhima World School** for providing the computer laboratory and all the facilities needed to write and test these programs.

I also thank the laboratory assistant for keeping the systems ready during every practical period, my parents for their support and motivation, and my friends for their help whenever I was stuck while writing a program or an SQL query.

Thank you for reading.

Aditya Shakya
Class XI - A, Section A, Roll No. 2

INDEX

No.	Topic	Page
PART A : PYTHON PROGRAMMING		
1	Interactive Mode Execution	4
2	Script Mode Execution	4
3	Python Keywords and Identifiers	4
4	Dynamic Typing and Variable Assignment	4
5	Standard Data Types	5
6	Input and Output Statements	5
7	Explicit Type Conversion (Typecasting)	5
8	Arithmetic Operators	5
9	Relational and Logical Operators	6
10	Conditional Constructs (if, if-else)	6
11	Multi-way Conditional Branching (if-elif-else)	6
12	The range() Function	6
13	Iteration with for Loop	7
14	Iteration with while Loop	7
15	String Operations and Index Slicing	7
16	Lists Manipulation	7
17	Dictionaries (Key-Value Mapping)	8
PART B : STRUCTURED QUERY LANGUAGE (SQL)		
18	CREATE TABLE - Creating a Table	8
19	DROP TABLE - Deleting a Table	8
20	INSERT INTO - Adding Records	9
21	UPDATE - Modifying Records	9
22	DELETE - Removing Records	9
23	SELECT - Retrieving Data	10

BIBLIOGRAPHY

1. **Informatics Practices, Class XI** - NCERT and CBSE textbook, New Delhi.
2. **Python Documentation** - official documentation of the Python language, docs.python.org.
3. **MySQL 8.0 Reference Manual** - Oracle Corporation, dev.mysql.com/doc.
4. **W3Schools SQL Tutorial** - online reference for SQL commands, www.w3schools.com/sql.
5. **Class notes and laboratory manual** - instructions given by the subject teacher.
6. **GitHub repository** - github.com/zionixxdev/xi-ip-practicals, where the programs of this file are kept.

PRACTICAL 1 Interactive Mode Execution

Aim : To execute Python statements in the interactive mode of the interpreter.

```
>>> 25 + 17
42
>>> 2 ** 10
1024
>>> 17 // 5
3
>>> 17 % 5
2
>>> 7 / 2
3.5
>>> len("Informatics")
11
>>> "Py" + "thon"
'Python'
>>> 15 > 9 and 4 < 2
False
```

```
Each statement was evaluated by the interpreter as soon as the Enter key was pressed and
the result was displayed on the line below it, as shown in the session above. No print()
statement and no program file are needed in interactive mode.
```

Conclusion : Every statement typed at the >>> prompt was evaluated at once and its value was displayed, which makes this mode useful for testing single expressions.

PRACTICAL 2 Script Mode Execution

Aim : To write a program in a file and execute it in script mode to find the area and the circumference of a circle.

```
radius = 7
area = 3.14159 * radius ** 2
circumference = 2 * 3.14159 * radius
print("Radius        :", radius)
print("Area          :", round(area, 2))
print("Circumference :", round(circumference, 2))
```

```
Radius        : 7
Area          : 153.94
Circumference : 43.98
```

Conclusion : The program was executed as a whole in script mode and the area and the circumference were displayed because print() statements were used.

PRACTICAL 3 Python Keywords and Identifiers

Aim : To display the keywords of Python and to check which of the given names are valid identifiers.

```
import keyword

print("Total keywords :", len(keyword.kwlist))
words = keyword.kwlist
for i in range(0, len(words), 12):
    print("  ", ", ".join(words[i:i + 12]))

names = ["student_name", "_marks", "class", "2total", "Roll-No"]
for name in names:
    if name in keyword.kwlist:
        print(name, "-> invalid : keyword")
    elif name[0].isdigit() or not name.isidentifier():
        print(name, "-> invalid : naming rule")
    else:
        print(name, "-> valid identifier")
```

```
Total keywords : 35
   False, None, True, and, as, assert, async, await, break, class, continue, def
   del, elif, else, except, finally, for, from, global, if, import, in, is
   lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield
student_name -> valid identifier
_marks -> valid identifier
class -> invalid : keyword
2total -> invalid : naming rule
Roll-No -> invalid : naming rule
```

Conclusion : All 35 keywords were listed and each given name was reported as a valid or an invalid identifier according to the rules of naming.

PRACTICAL 4 Dynamic Typing and Variable Assignment

Aim : To show that a variable in Python can refer to values of different data types at different times.

```
value = 25
print(value, type(value))
value = 25.75
print(value, type(value))
value = "Informatics"
print(value, type(value))
value = True
print(value, type(value))
value = [10, 20]
print(value, type(value))
```

```
25
25.75
Informatics
True
[10, 20]
```

Conclusion : The variable held an int, a float, a string, a bool and a list one after another, which proves that Python is dynamically typed.

PRACTICAL 5 Standard Data Types

Aim : To create variables of the standard data types of Python and to verify their types.

```
roll, marks, name, passed = 3, 87.5, "Aditya", True

print(roll, type(roll))
print(marks, type(marks))
print(name, type(name))
print(passed, type(passed))
print(2 + 3j, type(2 + 3j))
print("Sum :", roll + marks, "| Repeat :", "IP " * 3)
```

```
3
87.5
Aditya
True
(2+3j)
Sum : 90.5 | Repeat : IP IP IP
```

Conclusion : type() reported int, float, str, bool and complex for the values created, exactly as expected.

PRACTICAL 6 Input and Output Statements

Aim : To read data from the keyboard using input() and to display a report using print().

```
name = input("Name    : ")
roll = int(input("Roll No.: "))
eng = float(input("English : "))
maths = float(input("Maths   : "))
ip = float(input("IP      : "))

total = eng + maths + ip
print("Total   :", total, "| Average :", round(total / 3, 2))
print("Result  :", "PASS" if total / 3 >= 33 else "FAIL")
```

```
Name    : Aditya Shakya
Roll No.: 2
English : 78
Maths   : 85
IP      : 92
Total   : 255.0 | Average : 85.0
Result  : PASS
```

Conclusion : The data typed on the keyboard was converted into the required numeric types and the total, the average and the result were printed correctly.

PRACTICAL 7 Explicit Type Conversion (Typecasting)

Aim : To convert values from one data type to another using int(), float() and str().

```
a, b, c = "45", "3.14", 2025

print(int(a), type(int(a)))
print(float(b), type(float(b)))
print(str(c), type(str(c)))
print(int(float(b)), "-> fraction removed")
print(int(a) + 100)
print(bool(0), bool(1), bool(""), bool("IP"))
```

```
45
3.14
2025
3 -> fraction removed
145
False True False True
```

Conclusion : Explicit conversion changed strings into numbers and numbers into strings, and int() removed the decimal part of a float.

PRACTICAL 8 Arithmetic Operators

Aim : To perform all the arithmetic operations on two numbers using Python operators.

```
a, b = 17, 5

print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a // b =", a // b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
```

```
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a // b = 3
a % b  = 2
a ** b = 1419857
```

Conclusion : All seven operators gave the expected results and the difference between /, // and % was clearly seen.

PRACTICAL 9 Relational and Logical Operators

Aim : To evaluate relational and logical expressions in Python.

```
a, b, c = 10, 20, 15

print(a > b, a < b, a == c, a != c)
print(b >= c, b <= c)
print((a < b) and (b > c))
print((a > b) or (b > c))
print(not (a == c))
age = 17
print("Can vote :", age >= 18 and age <= 120)
```

```
False True False True
True False
True
True
True
Can vote : False
```

Conclusion : Every expression returned True or False as expected, and the combined condition rejected an age of 17 years for voting.

PRACTICAL 10 Conditional Constructs (if, if-else)

Aim : To take a decision in a program using the if and if-else constructs.

```
num = int(input("Enter a number : "))
if num % 2 == 0:
    print(num, "is even")
else:
    print(num, "is odd")

marks = float(input("Enter marks : "))
if marks >= 33:
    print("Result : PASS")
else:
    print("Result : FAIL")
```

```
Enter a number : 24
24 is even
Enter marks : 78
Result : PASS
```

Conclusion : The program followed the correct path in each case and identified an even number and a pass result.

PRACTICAL 11 Multi-way Conditional Branching (if-elif-else)

Aim : To assign a grade to a student on the basis of marks using the if-elif-else chain.

```
marks = float(input("Enter marks : "))

if marks >= 90:
    grade = "A1"
elif marks >= 80:
    grade = "A2"
elif marks >= 70:
    grade = "B1"
elif marks >= 33:
    grade = "C1"
else:
    grade = "E"

print("Marks :", marks, "| Grade :", grade)
```

```
Enter marks : 87
Marks : 87.0 | Grade : A2
```

Conclusion : Only one block of the chain was executed and the correct grade A2 was printed for 87 marks.

PRACTICAL 12 The range() Function

Aim : To study the working of range() by generating different sequences of numbers.

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(2, 11, 2)))
print(list(range(10, 0, -2)))
print("Sum 1 to 10 :", sum(range(1, 11)))
for i in range(1, 6):
    print(i ** 2, end=" ")
```

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[2, 4, 6, 8, 10]
[10, 8, 6, 4, 2]
Sum 1 to 10 : 55
1 4 9 16 25
```

Conclusion : range() produced the sequences 0 to 4, 1 to 5, the even numbers from 2 to 10 and a countdown as asked.

PRACTICAL 13 Iteration with for Loop

Aim : To print the multiplication table of a number, the sum of the first n natural numbers and the factorial of n using a for loop.

```
n = int(input("Enter a number : "))

for i in range(1, 6):
    print(n, "x", i, "=", n * i)

total = sum(range(1, n + 1))
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print("Sum 1 to", n, "=", total)
print("Factorial =", fact)
```

```
Enter a number : 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
Sum 1 to 6 = 21
Factorial = 720
```

Conclusion : The for loop repeated the block for every value of the range and produced the table, the sum 21 and the factorial 720.

PRACTICAL 14 Iteration with while Loop

Aim : To find the sum of the digits, the reverse and the number of digits of a number using a while loop.

```
n = 4729
temp, total, reverse, count = n, 0, 0, 0

while temp > 0:
    digit = temp % 10
    total += digit
    reverse = reverse * 10 + digit
    count += 1
    temp //= 10

print("Sum of digits :", total)
print("Reverse       :", reverse)
print("Number of digits :", count)
```

```
Sum of digits : 22
Reverse       : 9274
Number of digits : 4
```

Conclusion : The loop separated the digits one by one and gave the sum 22, the reverse 9274 and the digit count 4 for 4729.

PRACTICAL 15 String Operations and Index Slicing

Aim : To perform the common operations on a string and to extract substrings using slicing.

```
text = "Informatics Practices"

print("Length  :", len(text))
print("First   :", text[0], "| Last :", text[-1])
print("Upper   :", text.upper())
print("Slice   :", text[0:11], "|", text[12:])
print("Reverse :", text[::-1])
print("Step 2  :", text[::2])
print("Count a :", text.count("a"))
print("Replace :", text.replace("Practices", "Skills"))
print("Split   :", text.split())
```

```
Length  : 21
First   : I | Last : s
Upper   : INFORMATICS PRACTICES
Slice   : Informatics | Practices
Reverse : secitcarP scitamrofnI
Step 2  : IfraisPatcs
Count a : 2
Replace : Informatics Skills
Split   : ['Informatics', 'Practices']
```

Conclusion : The string was sliced, reversed and changed by the methods, while the original string remained unchanged.

PRACTICAL 16 Lists Manipulation

Aim : To create a list and to add, insert, remove, modify, sort and reverse its elements.

```
marks = [78, 85, 92, 66, 74]
print(marks)
print("Sum :", sum(marks), "Max :", max(marks), "Min :", min(marks))

marks.append(88)
print("append :", marks)
marks.insert(2, 95)
print("insert :", marks)
marks.remove(66)
print("remove :", marks)
marks.sort()
print("sort   :", marks)
```

```
[78, 85, 92, 66, 74]
Sum : 395 Max : 92 Min : 66
append : [78, 85, 92, 66, 74, 88]
insert : [78, 85, 95, 92, 66, 74, 88]
remove : [78, 85, 95, 92, 74, 88]
sort   : [74, 78, 85, 88, 92, 95]
```

Conclusion : Every operation changed the list itself, which shows that lists are mutable, unlike strings.

PRACTICAL 17 Dictionaries (Key-Value Mapping)

Aim : To create a dictionary of a student's record and to add, update and delete its items.

```
student = {"Roll No": 2, "Name": "Aditya", "Marks": 87}

print(student)
print("Name :", student["Name"], "| Items :", len(student))

student["Class"] = "XI-A"
student["Marks"] = 91
print("After add and update :", student)

del student["Class"]
print("After deleting Class :", student)

for key, value in student.items():
    print("  ", key, ":", value)
```

```
{'Roll No': 2, 'Name': 'Aditya', 'Marks': 87}
Name : Aditya | Items : 3
After add and update : {'Roll No': 2, 'Name': 'Aditya', 'Marks': 91, 'Class': 'XI-A'}
After deleting Class : {'Roll No': 2, 'Name': 'Aditya', 'Marks': 91}
   Roll No : 2
   Name : Aditya
   Marks : 91
```

Conclusion : The dictionary accepted a new key, updated a value and removed a key, and items() displayed every pair.

PRACTICAL 18 CREATE TABLE - Creating a Table

Aim : To create a table named student in MySQL using the CREATE TABLE command with suitable data types and a PRIMARY KEY constraint.

```
CREATE DATABASE IF NOT EXISTS school;
USE school;

CREATE TABLE student (
    adm_no INT PRIMARY KEY,
    name   VARCHAR(30) NOT NULL,
    class  VARCHAR(5),
    marks  INT,
    city   VARCHAR(20)
);

DESC student;
```

```
Query OK, 1 row affected (0.01 sec)

Database changed
Query OK, 0 rows affected (0.02 sec)

+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| adm_no | int         | NO   | PRI | NULL    |       |
| name   | varchar(30) | NO   |     | NULL    |       |
| class  | varchar(5)  | YES  |     | NULL    |       |
| marks  | int         | YES  |     | NULL    |       |
| city   | varchar(20) | YES  |     | NULL    |       |
+--------+-------------+------+-----+---------+-------+
5 rows in set
```

Conclusion : The table student was created with adm_no as the primary key and DESC confirmed its structure.

PRACTICAL 19 DROP TABLE - Deleting a Table

Aim : To delete an existing table from the database using the DROP TABLE command.

```
USE school;

CREATE TABLE temp_data (id INT, item VARCHAR(20));
INSERT INTO temp_data VALUES (1, 'Pen'), (2, 'Book');

SELECT * FROM temp_data;

DROP TABLE temp_data;

SHOW TABLES;
```

```
Database changed
Query OK, 0 rows affected (0.03 sec)
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

+----+------+
| id | item |
+----+------+
| 1  | Pen  |
| 2  | Book |
+----+------+
2 rows in set

Query OK, 0 rows affected (0.02 sec)

+------------------+
| Tables_in_school |
+------------------+
| student          |
+------------------+
1 row in set
```

Conclusion : The table temp_data and its two records were removed completely and only the student table was left in the database.

PRACTICAL 20 INSERT INTO - Adding Records

Aim : To insert new rows into the student table using the INSERT INTO command.

```
USE school;

INSERT INTO student VALUES (1, 'Aditya Shakya', 'XI-A', 87, 'Lucknow');
INSERT INTO student VALUES (2, 'Priya Verma', 'XI-A', 92, 'Kanpur');
INSERT INTO student VALUES (3, 'Rahul Singh', 'XI-B', 75, 'Agra');
INSERT INTO student (adm_no, name, class, marks)
       VALUES (4, 'Neha Gupta', 'XI-B', 81);
INSERT INTO student VALUES (5, 'Aman Tiwari', 'XI-A', 68, 'Lucknow');

SELECT * FROM student;
```

```
Database changed
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)

+--------+--------------+-------+-------+---------+
| adm_no | name         | class | marks | city    |
+--------+--------------+-------+-------+---------+
| 1      | Aditya Shakya | XI-A  | 87    | Lucknow |
| 2      | Priya Verma   | XI-A  | 92    | Kanpur  |
| 3      | Rahul Singh   | XI-B  | 75    | Agra    |
| 4      | Neha Gupta    | XI-B  | 81    | NULL    |
| 5      | Aman Tiwari   | XI-A  | 68    | Lucknow |
+--------+--------------+-------+-------+---------+
5 rows in set
```

Conclusion : Five records were inserted and the row for which the city was not given contained NULL.

PRACTICAL 21 UPDATE - Modifying Records

Aim : To modify the values already stored in a table using the UPDATE command with SET and WHERE.

```
USE school;

UPDATE student SET marks = 90 WHERE adm_no = 1;
UPDATE student SET city = 'Varanasi' WHERE adm_no = 4;
UPDATE student SET marks = marks + 5 WHERE class = 'XI-B';

SELECT * FROM student;
```

```
Database changed
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0

+--------+--------------+-------+-------+----------+
| adm_no | name         | class | marks | city     |
+--------+--------------+-------+-------+----------+
| 1      | Aditya Shakya | XI-A  | 90    | Lucknow  |
| 2      | Priya Verma   | XI-A  | 92    | Kanpur   |
| 3      | Rahul Singh   | XI-B  | 80    | Agra     |
| 4      | Neha Gupta    | XI-B  | 86    | Varanasi |
| 5      | Aman Tiwari   | XI-A  | 68    | Lucknow  |
+--------+--------------+-------+-------+----------+
5 rows in set
```

Conclusion : Only the rows that satisfied the WHERE condition were changed, which shows why a correct condition is important.

PRACTICAL 22 DELETE - Removing Records

Aim : To remove selected rows from a table using the DELETE command with the WHERE clause.

```
USE school;

DELETE FROM student WHERE adm_no = 5;
DELETE FROM student WHERE city = 'Kanpur';

SELECT * FROM student;
```

```
Database changed
Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.01 sec)

+--------+--------------+-------+-------+----------+
| adm_no | name         | class | marks | city     |
+--------+--------------+-------+-------+----------+
| 1      | Aditya Shakya | XI-A  | 90    | Lucknow  |
| 3      | Rahul Singh   | XI-B  | 80    | Agra     |
| 4      | Neha Gupta    | XI-B  | 86    | Varanasi |
+--------+--------------+-------+-------+----------+
3 rows in set
```

Conclusion : Exactly the two rows that satisfied the conditions were removed and the remaining three records were untouched.

PRACTICAL 23 SELECT - Retrieving Data

Aim : To retrieve data from a table using SELECT with conditions, sorting and aggregate functions.

```
USE school;

SELECT * FROM student;

SELECT name, marks FROM student
       WHERE marks >= 85 ORDER BY marks DESC;

SELECT COUNT(*) AS total, AVG(marks) AS average,
       MAX(marks) AS highest, MIN(marks) AS lowest
FROM student;

SELECT class, COUNT(*) AS students FROM student GROUP BY class;
```

```
Database changed

+--------+---------------+-------+-------+----------+
| adm_no | name          | class | marks | city     |
+--------+---------------+-------+-------+----------+
| 1      | Aditya Shakya | XI-A  | 90    | Lucknow  |
| 3      | Rahul Singh   | XI-B  | 80    | Agra     |
| 4      | Neha Gupta    | XI-B  | 86    | Varanasi |
+--------+---------------+-------+-------+----------+
3 rows in set

+---------------+-------+
| name          | marks |
+---------------+-------+
| Aditya Shakya | 90    |
| Neha Gupta    | 86    |
+---------------+-------+
2 rows in set


+-------+---------+---------+--------+
| total | average | highest | lowest |
+-------+---------+---------+--------+
| 3     | 85.3333 | 90      | 80     |
+-------+---------+---------+--------+
1 row in set

+-------+----------+
| class | students |
+-------+----------+
| XI-A  | 1        |
| XI-B  | 2        |
+-------+----------+
2 rows in set
```

Conclusion : The SELECT command retrieved complete rows, selected columns, filtered and sorted rows, aggregate values and grouped counts without changing the table.